NeupBase

NeupBase is an AI-powered Academic Document Vault and Dynamic Study Scheduler. It leverages advanced Large Language Models (LLMs) to automatically parse academic documents, structure syllabuses, and generate personalized, micro-task-based study plans for students.

🚀 Project Overview

The project is designed to eliminate the friction of organizing study materials. By uploading course documents (like syllabuses or lecture slides), NeupBase extracts the content, breaks it down into logical topics, and dynamically schedules 30-60 minute study sessions. It tracks your completion continuously, adjusting the schedule as needed and providing analytical preparation scores for exams.

🏗️ Architecture & Sections

The repository is organized into four main components:

1. `dynamic_scheduler` (Frontend Application)

The user-facing web application and primary backend for client interactions.

- **Role:** Web interface, task management, PWA delivery, and AI chatbot interactions.
- **Tech Stack:** Next.js 16 (App Router), React 19, Tailwind CSS v4, Prisma ORM, Supabase SSR/Auth, Vercel AI SDK.
- **Key Features:**
  - Progressive Web App (PWA) with push notifications.
  - Data visualizations using Recharts.
  - Task visualization and state management.

2. `python_pipeline` (Document Ingestion Engine)

An asynchronous API pipeline responsible for parsing raw academic files.

- **Role:** Extracts text from uploaded files and uses AI to structure the content into topics.
- **Tech Stack:** FastAPI, Python, `pdfplumber`, `python-docx`, `python-pptx`, Google GenAI (Gemini 3.5 Flash).
- **Key Features:**
  - Webhook-based background processing.
  - Resilience checks (e.g., password protection, OCR requirement detection).

- Direct integration with Supabase to update document metadata.

3. `semester_planner` (AI Study Planner)

The core intelligence engine for generating actionable study plans.

- **Role:** Decomposes high-level topics into discrete micro-tasks.
- **Tech Stack:** Python, Google GenAI SDK (Gemini 3.1 Pro), Pydantic, PostgreSQL client (`psycopg2`).
- **Key Features:**
  - Strict JSON schema adherence for task generation.
  - `ai_generator` : Creates 30-60 minute micro-tasks (read, solve, summarize) based on lecture estimates.
  - `weekly_packager` & `rollover_engine` : Manages the continuous flow of tasks across weeks.

4. `supabase` (Database & Backend-as-a-Service)

The foundational database and authentication layer.

- **Role:** Secure, scalable data storage and real-time state syncing.
- **Tech Stack:** PostgreSQL, Supabase Auth, Row Level Security (RLS).
- **Key Features:**
  - Append-only task completion logs for tamper-proof telemetry.
  - Advanced indexing (BRIN, B-Tree covering indices) for time-series and state retrieval.
  - Realtime WebSocket updates via PostgreSQL logical replication.
  - RPC functions for on-the-fly analytical scoring (e.g., exam prep scores).

🛠️ Languages, Tools, & Components

- **Languages:** TypeScript, Python, SQL, JavaScript, HTML/CSS.
- **Frameworks:** Next.js, FastAPI, Tailwind CSS.
- **AI Providers:** Google (Gemini 3.1 Pro & 3.5 Flash).
- **Database:** Supabase (PostgreSQL), Prisma.
- **Utilities:** `rrule` (for recurring tasks), `mammoth` (docx parsing), `recharts` (analytics).

💻 How to Use and Operate Daily

To run the full stack locally for development or daily operation, follow these steps:

Prerequisites

- Node.js (v18+)
- Python (v3.10+)
- Supabase CLI (if running DB locally) or a Supabase Cloud instance.
- API Keys for Google Gemini (supporting Gemini 3.1 Pro and 3.5 Flash).

1. Database Setup

Ensure your Supabase instance is running and the database schemas (located in `dynamic_scheduler/supabase_*_schema.sql` and `supabase/migrations/`) have been applied. Set the necessary connection strings in the `.env` files for both Next.js and Python services.

2. Start the Document Ingestion Pipeline

Open a terminal and navigate to the `python_pipeline` directory:

```
cd python_pipeline
pip install -r requirements.txt
# Start the FastAPI server (runs on port 8000 by default)
uvicorn app.main:app --reload
```

3. Start the Frontend Scheduler

Open a second terminal and navigate to the `dynamic_scheduler` directory:

```
cd dynamic_scheduler
npm install
# Start the Next.js development server
npm run dev
```

The web app will be available at `http://localhost:3000`.

4. Running the Semester Planner Engines

The `semester_planner` scripts are meant to be invoked as cron jobs or triggered via backend tasks to generate tasks, package weekly schedules, and roll over incomplete tasks. You can run them manually:

```
cd semester_planner
pip install -r requirements.txt
# Example: Run the rollover engine at the end of the week
python -m engine.rollover_engine
```

Daily Workflow

1. **Upload Syllabus/Docs:** Use the web interface to upload course documents. The Next.js app sends these to Supabase Storage and triggers the `python_pipeline` webhook.
2. **AI Processing:** The pipeline extracts text, structures topics via Gemini, and saves them back to the database.
3. **Task Generation:** The `semester_planner` (Gemini) detects new topics and generates a micro-task schedule.
4. **Study & Complete:** Open the PWA/Web app to view your daily brief, complete tasks, and track your prep score in real-time.